



Cyberscope

A *TAC Security* Company

Audit Report

Nummus

June 2026

Network POL

Address 0x53688855f97Ed8b119d3F6AE0ffac6a5E165992e

Audited by © cyberscope

Table of Contents

Table of Contents	1
Analysis	3
Diagnostics	4
Risk Classification	5
Review	6
Audit Updates	6
Source Files	6
Findings Breakdown	7
ST - Stops Transactions	8
Description	8
Recommendation	9
Team Update	10
CCR - Contract Centralization Risk	12
Description	12
Recommendation	13
Team Update	14
IRC - Ineffective Reference Call	15
Description	15
Recommendation	15
Team Update	15
ISP - Insufficient Slippage Protection	16
Description	16
Recommendation	16
Team Update	17
MEM - Missing Error Messages	18
Description	18
Recommendation	18
Team Update	18
PLPI - Potential Liquidity Provision Inadequacy	19
Description	19
Recommendation	19
Team Update	20
RD - Redundant Declaration	21
Description	21
Recommendation	21
Team Update	21
RED - Redundant Event Declaration	22
Description	22
Recommendation	22

Team Update	22
RSK - Redundant Storage Keyword	23
Description	23
Recommendation	23
Team Update	23
SCA - Suboptimal Contract Architecture	24
Description	24
Recommendation	25
Team Update	25
UFA - Untracked Fee Accumulation	26
Description	26
Recommendation	26
Team Update	26
L07 - Missing Events Arithmetic	27
Description	27
Recommendation	27
Team Update	27
L14 - Uninitialized Variables in Local Scope	28
Description	28
Recommendation	28
Team Update	28
L16 - Validate Variable Setters	29
Description	29
Recommendation	29
Team Update	29
Functions Analysis	30
Inheritance Graph	36
Flow Graph	37
Summary	38
Disclaimer	39
About Cyberscope	40

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Acknowledged
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	CCR	Contract Centralization Risk	Acknowledged
●	IRC	Ineffective Reference Call	Acknowledged
●	ISP	Insufficient Slippage Protection	Acknowledged
●	MEM	Missing Error Messages	Acknowledged
●	PLPI	Potential Liquidity Provision Inadequacy	Acknowledged
●	RD	Redundant Declaration	Acknowledged
●	RED	Redundant Event Declaration	Acknowledged
●	RSK	Redundant Storage Keyword	Acknowledged
●	SCA	Suboptimal Contract Architecture	Acknowledged
●	UFA	Untracked Fee Accumulation	Acknowledged
●	L07	Missing Events Arithmetic	Acknowledged
●	L14	Uninitialized Variables in Local Scope	Acknowledged
●	L16	Validate Variable Setters	Acknowledged

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	Nummus
Compiler Version	v0.8.35+commit.47b9dedd
Optimization	200 runs
Explorer	https://polygonscan.com/token/0x53688855f97ed8b119d3f6ae0ffac6a5e165992e
Address	0x53688855f97Ed8b119d3F6AE0ffac6a5E165992e
Network	POL
Symbol	\$NUMMUS
Decimals	18
Total Supply	400,000,000

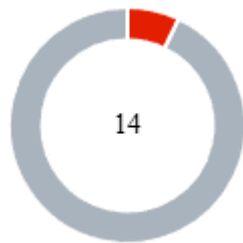
Audit Updates

Initial Audit	15 Jun 2026 https://github.com/cyberscope-io/audits/blob/main/nummus/v1/audit.pdf
Corrected Phase 2	29 Jun 2026

Source Files

Filename	SHA256
Nummus.sol	082a67ec8ba7d38d92570a9925aced8e0b2f64b3781c078e017a2c9d634fc513

Findings Breakdown



● Critical	1
● Medium	0
● Minor / Informative	13

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	1	0	0
● Medium	0	0	0	0
● Minor / Informative	0	13	0	0

ST - Stops Transactions

Criticality	Critical
Location	Nummus.sol#L1109,1134,1407,1573,1576
Status	Acknowledged

Description

Initially, the contract does not allow the non-excluded addresses to transfer tokens. The restriction can be resumed once the contract owner enables them.

```
if (!canTransferBeforeTradingIsEnabled[from]) {  
    require(tradingEnabled, "Trading has not yet been enabled");  
}
```

The contract owner grants pre-trading access to arbitrary addresses through the `setCanTransferBefore` function, and the `setPresaleWallet` function grants the same access. The restriction is enforced on the sender of the transaction. As a result, the whitelisted addresses and the pair are able to trade during the restricted period while regular users remain blocked from selling or transferring their tokens. This creates a privileged trading window where selected addresses can interact with the pair before the wider user base is able to trade.

```
function setCanTransferBefore(  
    address wallet,  
    bool enable  
) external onlyOwner {  
    canTransferBeforeTradingIsEnabled[wallet] = enable;  
}
```

The contract distributes the collected fees to four recipient wallets through low-level calls within `swapAndSendDividends`, and these calls forward all remaining gas without a gas stipend limit. Each call captures its result in a success flag, so a recipient that reverts on receipt returns `false` and the distribution proceeds without disruption. However, because no gas limit is placed on these calls, a recipient that consumes an excessive amount of gas can drain the gas of the calling transaction. Since `swapAndSendDividends` is reached from within `_transfer` through the automatic

swap trigger, such a recipient can cause the transfer that triggered the distribution to revert, halting transfers under this condition.

```
if (marketingPayout > 0) {
    (successOp, ) = address(MarketingWallet).call{
        value: marketingPayout
    }("");
    success = success && successOp;
}
```

The `swapAndLiquify` function does not guard against a zero swap amount. The owner can set the `swapTokensAtAmount` to zero or even a very small number, and as a result the contract will try to swap zero tokens. The function computes `half = tokens.div(2)` and swaps that half; under certain fee and balance conditions the computed swap amount can round down such that `half` is zero, in which case the underlying swap is invoked with zero tokens and reverts on the router. As this runs inside the transfer flow without a `try/catch`, the triggering transfer would revert.

```
function swapAndLiquify(uint256 tokens) private {
    uint256 half = tokens.div(2);
    uint256 otherHalf = tokens.sub(half);
    uint256 initialBalance = address(this).balance;
    swapTokensForEth(half);
    ...
}
```

Additionally, as mentioned in the `SCA` finding, the gas price limit enforced within `_transfer` can cause transactions to revert while the limits are active, which can likewise result in transfers being stopped.

Recommendation

The contract could embody a check to ensure that trading, once enabled, cannot be selectively restricted for regular users while privileged addresses retain access. The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

Team Update

1. **Pre-trading restriction:** This is the intended presale mechanism. Trading is enabled exactly once via `enableTrading()`, which is one-way by construction — the function requires `!tradingEnabled` and sets it to `true;` the contract contains no code path that can set `tradingEnabled` back to `false`. Once trading is enabled at launch, transfers can never again be restricted by this mechanism.
2. **Pre-trading whitelist:** `setCanTransferBefore` and `setPresaleWallet` exist solely to operate the presale (distribution and liquidity preparation) and become permanently irrelevant once trading is enabled.
3. **Unlimited-gas fee distribution:** The four fee recipient wallets are hardcoded in the constructor and are externally owned accounts. No setter function exists to change them. Consequently, the theoretical gas-exhaustion scenario requires a malicious contract recipient, which cannot occur with the deployed configuration.
4. **Zero-amount swap edge case:** The swap trigger threshold is enforced with an on-chain minimum floor, making the zero-half rounding path practically unreachable under normal operation.
5. **Launch limits (gas price, cooldown, per-block):** These apply only while `limitsInEffect` is active and exist solely as anti-sniper and anti-bot protection during the launch window — a standard practice for fair launches. The team will disable limits following launch stabilization. We further note that the protections are hard-bounded on-chain: the sell cooldown is capped at 5 minutes (`setcooldowntimer` enforces `value <= 5 minutes`), the maximum wallet cannot be set below 0.05% of supply, and fees are capped by immutable limits. Token sales can therefore be throttled during the protected window but can never be blocked. We acknowledge that, per your methodology, the permanent resolution is

ownership renouncement, which the team will evaluate post-launch once operational configuration (limits, fee finalization) is complete.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	Nummus.sol#L1101,1109,1117,1123,1128,1134,1142,1147,1153,1159,1191,1195,1201,1208,1212,1226,1232,1238,1255,1380,1596,1607,1613,1632
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion. The owner has the authority to designate or remove any address as an automated market maker pair through `setAutomatedMarketMakerPair`, which determines whether a transfer is taxed as a buy or a sell, may hinder the standard operational workflow. Additionally, the owner has the authority to set the token in which dividends are paid to any address through `updatePayoutToken`, without validation. As a result, the payout can be pointed to a non-standard or illiquid token, causing the dividend swap to return an unexpected amount or to fail.

```
function enableTrading() external onlyOwner {...}
function setPresaleWallet(address wallet) external onlyOwner {...}
function setExcludeFees(address account, bool excluded) public onlyOwner
{...}
function setExcludeDividends(address account) public onlyOwner {...}
function setIncludeDividends(address account) public onlyOwner {...}
function setCanTransferBefore(address wallet, bool enable) external
onlyOwner {...}
function setLimitsInEffect(bool value) external onlyOwner {...}
function setGasPriceLimit(uint256 maxGwei) external onlyOwner {...}
function setcooldowntimer(uint256 value) external onlyOwner {...}
function setmaxWallet(uint256 value) external onlyOwner {...}
function setSwapTriggerAmount(uint256 amount) public onlyOwner {...}
function enableSwapAndLiquify(bool enabled) public onlyOwner {...}
function setAutomatedMarketMakerPair(address pair, bool value) public
onlyOwner {...}
function setAllowCustomTokens(bool allow) public onlyOwner {...}
function setAllowAutoReinvest(bool allow) public onlyOwner {...}
function updateGasForProcessing(uint256 newValue) public onlyOwner {...}
function transferAdmin(address newOwner) public onlyOwner {...}
function updateTransferFee(uint256 newTransferFee) public onlyOwner {...}
function updateFees(uint256 marketingBuy, uint256 marketingSell, uint256
teamBuy, uint256 teamSell, uint256 team2FundsBuy, uint256 team2FundsSell,
uint256 liquidityBuy, uint256 liquiditySell, uint256 team3Buy, uint256
team3Sell) public onlyOwner {...}
function setDividendsPaused(bool value) external onlyOwner {...}
function updatePayoutToken(address token) public onlyOwner {...}
function setMinimumTokenBalanceForAutoDividends(uint256 value) public
onlyOwner {...}
function setMinimumTokenBalanceForDividends(uint256 value) public
onlyOwner {...}
function forceSwapAndSendDividends(uint256 tokens) public onlyOwner {...}
function setAutomatedMarketMakerPair(address pair, bool value) public
onlyOwner{...}
function updatePayoutToken(address token) public onlyOwner{...}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

The team has acknowledged that this is not a security issue and states:

The owner functions listed are required for launch operations (presale management, anti-bot limits, and fee configuration). We note that fee configuration is bounded by immutable on-chain caps, and trading enablement is irreversible. Owner keys are managed under strict internal key-management practices, and ownership renouncement will be evaluated post-launch.

IRC - Ineffective Reference Call

Criticality	Minor / Informative
Location	Nummus.sol#L1059
Status	Acknowledged

Description

The contract intends to exclude the zero address from receiving dividends during the constructor. The statement references the `excludedFromDividends` public mapping getter instead of the `excludeFromDividends` function. As a result, the call returns the stored value and discards it, and the zero address is not added to the excluded set. The remaining exclusions in the constructor use the correct function.

```
dividendTracker.excludedFromDividends(address(0));
```

Recommendation

The team is advised to call the `excludeFromDividends` function so that the intended exclusion is applied.

Team Update

The team has acknowledged that this is not a security issue and states:

We confirm the constructor statement references the `excludedFromDividends` mapping getter rather than the `excludeFromDividends` function, making the call a no-op for the zero address. We note there is no functional impact: the ERC-20 implementation prevents the zero address from ever holding a balance, so it can never accrue dividends. The correction will be incorporated in any future contract iteration.

ISP - Insufficient Slippage Protection

Criticality	Minor / Informative
Location	Nummus.sol#L1589
Status	Acknowledged

Description

The contract swaps the accumulated fee tokens for the native currency during the fee distribution while passing zero as the minimum output amount. Through `swapAndSendDividends`, the `swapExactTokensForETHSupportingFeeOnTransferTokens` function is invoked with a minimum output of zero, so the swap accepts any resulting amount. This path is reached automatically inside `_transfer` and manually when the owner calls `forceSwapAndSendDividends`. As a result, the swap is exposed to sandwich attacks and unfavorable price movements.

```
uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(  
    tokenAmount,  
    0, // accept any amount of ETH  
    path,  
    address(this),  
    block.timestamp  
);
```

Recommendation

The team is advised to calculate a minimum output amount based on a tolerated slippage percentage, so that the swap reverts when the received amount falls below the acceptable threshold. Within the automatic transfer flow this protection is unavoidable to some extent, as a reverting swap would block the transfer; however, it could be applied to the owner-triggered `forceSwapAndSendDividends`, where a reverting swap does not affect user transfers.

Team Update

The team has acknowledged that this is not a security issue and states:

As noted in the report itself, slippage protection within the automatic transfer flow is unavoidable to a large extent, as a reverting swap would block user transfers. Fee swaps are small and frequent, limiting price-impact exposure. Slippage parameters for the owner-triggered `forceSwapAndSendDividends` path will be considered for any future iteration.

MEM - Missing Error Messages

Criticality	Minor / Informative
Location	Nummus.sol#L526,534
Status	Acknowledged

Description

The contract is missing error messages. Specifically, there are no error messages to accurately reflect the problem, making it difficult to identify and fix the issue. As a result, the users will not be able to find the root cause of the error.

```
require(b >= 0)
require(a >= 0)
```

Recommendation

The team is suggested to provide a descriptive message to the errors. This message can be used to provide additional context about the error that occurred or to explain why the contract execution was halted. This can be useful for debugging and for providing more information to users that interact with the contract.

Team Update

The team has acknowledged that this is not a security issue and states:

The referenced `require` statements are internal type-cast invariant guards that cannot fail under normal operation. Descriptive messages will be added in any future iteration.

PLPI - Potential Liquidity Provision Inadequacy

Criticality	Minor / Informative
Location	Nummus.sol#L1572,1582,1617
Status	Acknowledged

Description

The contract operates under the assumption that liquidity is consistently provided to the pair between the contract's token and the native currency. However, there is a possibility that liquidity is provided to a different pair. This inadequacy in liquidity provision in the main pair could expose the contract to risks. Specifically, during eligible transactions, where the contract attempts to swap tokens with the main pair, a failure may occur if liquidity has been added to a pair other than the primary one. Consequently, transactions triggering the swap functionality will result in a revert.

```
function swapAndLiquify(uint256 tokens) private {
    uint256 half = tokens.div(2);
    uint256 otherHalf = tokens.sub(half);
    uint256 initialBalance = address(this).balance;
    swapTokensForEth(half);
    uint256 newBalance = address(this).balance.sub(initialBalance);
    addLiquidity(otherHalf, newBalance);
    emit SwapAndLiquify(half, newBalance, otherHalf);
}
```

Recommendation

The team is advised to implement a runtime mechanism to check if the pair has adequate liquidity provisions. This feature allows the contract to omit token swaps if the pair does not have adequate liquidity provisions, significantly minimizing the risk of potential failures.

Furthermore, the team could ensure the contract has the capability to switch its active pair in case liquidity is added to another pair.

Additionally, the contract could be designed to tolerate potential reverts from the swap functionality, especially when it is a part of the main transfer flow. This can be achieved by

executing the contract's token swaps in a non-reversible manner, thereby ensuring a more resilient and predictable operation.

Team Update

The team has acknowledged that this is not a security issue and states:

The team provisions and maintains liquidity exclusively on the primary token/native pair, and the swap functionality only becomes active once launch liquidity is in place. Liquidity operations are managed to ensure the primary pair remains adequately provisioned.

RD - Redundant Declaration

Criticality	Minor / Informative
Location	Nummus.sol#L17
Status	Acknowledged

Description

The contract declares the `IUniswapV2Pair` interface, but the interface is not referenced anywhere in the contract. As a result, the declaration is redundant and increases the size of the source without contributing to the functionality.

```
interface IUniswapV2Pair {  
    ...  
}
```

Recommendation

The team is advised to remove the unused interface declaration so that the source contains only the definitions that are used by the contract.

Team Update

The team has acknowledged that this is not a security issue and states:

The unused `IUniswapV2Pair` interface is declaration-only, with no bytecode, runtime, or security impact. It will be removed in any future iteration.

RED - Redundant Event Declaration

Criticality	Minor / Informative
Location	Nummus.sol#L958,1738
Status	Acknowledged

Description

There are code segments that could be optimized. A segment may be optimized so that it becomes a smaller size, consumes less memory, executes more rapidly, or performs fewer operations.

The contract declares the events `UpdateUniswapV2Router` and `ClaimWaitUpdated` but they are not being used in the contract. As a result, they are redundant.

```
event UpdateUniswapV2Router(  
    address indexed newAddress,  
    address indexed oldAddress  
);  
  
event ClaimWaitUpdated(uint256 indexed newValue, uint256 indexed  
oldValue);
```

Recommendation

The team is advised to take these segments into consideration and rewrite them so the runtime will be more performant. That way it will improve the efficiency and performance of the source code and reduce the cost of executing it.

Team Update

The team has acknowledged that this is not a security issue and states:

The unused event declarations (`UpdateUniswapV2Router` , `ClaimWaitUpdated`) have no runtime or security impact and will be removed in any future iteration.

RSK - Redundant Storage Keyword

Criticality	Minor / Informative
Location	Nummus.sol#L2159,2164,2174,2180
Status	Acknowledged

Description

The contract uses the `storage` keyword in a view function. The `storage` keyword is used to persist data on the contract's storage. View functions are functions that do not modify the state of the contract and do not perform any actions that cost gas (such as sending a transaction). As a result, the use of the `storage` keyword in view functions is redundant.

```
Map storage map
```

Recommendation

It is generally considered good practice to avoid using the `storage` keyword in view functions, because it is unnecessary and can make the code less readable.

Team Update

The team has acknowledged that this is not a security issue and states:

The `Map storage` parameters appear in internal library functions operating on struct references, where the `storage` data location is required Solidity syntax. No impact on behavior or gas.

SCA - Suboptimal Contract Architecture

Criticality	Minor / Informative
Location	Nummus.sol#L1465,1465,2113
Status	Acknowledged

Description

The contract contains several design choices that deviate from best practices. These are described below. The native dividend payout forwards a hardcoded `gas: 3000` stipend through `user.call`. This is sufficient for an externally owned account but not for a smart-contract or multi-signature wallet that writes storage on receipt; such a recipient is not paid on the native path, though its dividend is preserved and remains claimable.

```
(success, ) = user.call({
    value: _withdrawableDividend,
    gas: 3000
}) ("");
```

The transfer cooldown and one-transaction-per-block limits are keyed on `tx.origin` rather than on the actual token holder participating in the transfer. This is incorrect because `tx.origin` may differ from the account that owns and transfers the tokens, such as when transfers are executed through a router, smart wallet, or other intermediary contract. The limits should therefore be tracked against the from address instead.

```
require(
    _holderLastTransferBlock[tx.origin] != block.number,
    "Too many TX in block"
);
```

The gas price limit relies on a hardcoded default of `7 gwei`, and while the owner can adjust it, the setter enforces a minimum of `5 gwei`. While limits are active, this can cause transactions to revert during periods of network congestion until the owner adjusts it.

```
require(
    tx.gasprice <= gasPriceLimit,
    "Gas price exceeds limit."
);
```

Recommendation

The team is advised to forward an adjustable gas stipend on the native payout path so smart-contract wallets can be served, to key the transfer limits on the participating account instead of `tx.origin`, and to avoid leaving the gas price limit at a level that can obstruct transactions during network congestion. These observations do not by themselves place user funds at risk.

Team Update

The team has acknowledged that this is not a security issue and states:

1. The 3,000-gas stipend on the native dividend payout intentionally serves EOA recipients while preventing reentrancy and gas-griefing; as the report notes, contract-wallet recipients retain their full dividend as claimable.
2. The `tx.origin`-keyed cooldown and per-block limits apply only while `limitsInEffect` is active as temporary anti-bot launch protection and will be disabled thereafter. The cooldown is hard-capped at 5 minutes on-chain, so transfers can be throttled but never blocked.
3. The gas price limit is likewise a temporary anti-sniper launch protection, adjustable by the owner and disabled together with the limits. We note the report's own conclusion that these observations do not place user funds at risk.

UFA - Untracked Fee Accumulation

Criticality	Minor / Informative
Location	Nummus.sol#L1422,1484
Status	Acknowledged

Description

The contract accumulates the collected buy and sell fees in the `buyAmount` and `sellAmount` variables and splits the swapped tokens according to these amounts. The transfer fee applied on wallet-to-wallet transfers and any tokens sent directly to the contract increase the contract balance without increasing `buyAmount` or `sellAmount`. Since the swap operates on the entire contract balance, these untracked tokens are included in the swap and distributed among the recipients.

```
super._transfer(from, address(this), tFees);  
...  
uint256 contractTokenBalance = balanceOf(address(this));
```

Recommendation

The team is advised to base the swap on the accumulated `buyAmount` and `sellAmount` rather than the entire contract balance, so that only the collected fees are processed.

Team Update

The team has acknowledged that this is not a security issue and states:

All tokens held by the contract are fee proceeds intended for distribution; including any incidental balance in the swap is intended behavior and results in no loss to holders. The wallet-to-wallet transfer fee is set to zero by default.

L07 - Missing Events Arithmetic

Criticality	Minor / Informative
Location	Nummus.sol#L1640
Status	Acknowledged

Description

Events are a way to record and log information about changes or actions that occur within a contract. They are often used to notify external parties or clients about events that have occurred within the contract, such as the transfer of tokens or the completion of a task.

It's important to carefully design and implement the events in a contract, and to ensure that all required events are included. It's also a good idea to test the contract to ensure that all events are being properly triggered and logged.

```
buyAmount = (buyAmount - fromBuy)
```

Recommendation

By including all required events in the contract and thoroughly testing the contract's functionality, the contract ensures that it performs as intended and does not have any missing events that could cause issues with its arithmetic.

Team Update

The team has acknowledged that this is not a security issue and states:

The accumulator adjustment occurs within an owner-only maintenance function that is monitored operationally; the automatic path emits the corresponding event. Additional event coverage will be considered in any future iteration.

L14 - Uninitialized Variables in Local Scope

Criticality	Minor / Informative
Location	Nummus.sol#L1401,1405,1635,1636
Status	Acknowledged

Description

Using an uninitialized local variable can lead to unpredictable behavior and potentially cause errors in the contract. It's important to always initialize local variables with appropriate values before using them.

```
uint256 marketingFees
uint256 teamFees
uint256 team2FundsFees
uint256 liquidityFee
uint256 team3Fees
uint256 fromBuy
uint256 fromSell
```

Recommendation

By initializing local variables before using them, the contract ensures that the functions behave as expected and avoid potential issues.

Team Update

The team has acknowledged that this is not a security issue and states:

Solidity zero-initializes local variables by default, and each referenced variable is assigned within the applicable branch prior to use. No behavioral impact.

L16 - Validate Variable Setters

Criticality	Minor / Informative
Location	Nummus.sol#L459,1762,2036
Status	Acknowledged

Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing of proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

```
_owner = msgSender  
defaultToken = token
```

Recommendation

By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

Team Update

The team has acknowledged that this is not a security issue and states:

Ownership assignment follows the standard Ownable pattern, and `updatePayoutToken` is an owner-only operational function. Zero-address validation will be added in any future iteration.

Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
Context	Implementation			
	_msgSender	Internal		
	_msgData	Internal		
DividendPayingTokenOptionalInterface	Interface			
	withdrawableDividendOf	External		-
	withdrawnDividendOf	External		-
	accumulativeDividendOf	External		-
DividendPayingTokenInterface	Interface			
	dividendOf	External		-
	distributeDividends	External	Payable	-
	withdrawDividend	External	✓	-
Ownable	Implementation	Context		
		Public	✓	-
	owner	Public		-

	renounceOwnership	Public	✓	onlyOwner
	transferOwnership	Public	✓	onlyOwner
DividendPaying Token	Implementation	ERC20, DividendPayingTokenInterface, DividendPayingTokenOptionalInterface		
		Public	✓	ERC20
	_toInt256Safe	Private		
	_toUInt256Safe	Private		
		External	Payable	-
	distributeDividends	Public	Payable	-
	withdrawDividend	Public	✓	-
	_withdrawDividendOfUser	Internal	✓	
	dividendOf	Public		-
	withdrawableDividendOf	Public		-
	withdrawnDividendOf	Public		-
	accumulativeDividendOf	Public		-
	_transfer	Internal	✓	
	_mint	Internal	✓	
	_burn	Internal	✓	
	_setBalance	Internal	✓	
Nummus	Implementation	ERC20, Ownable		

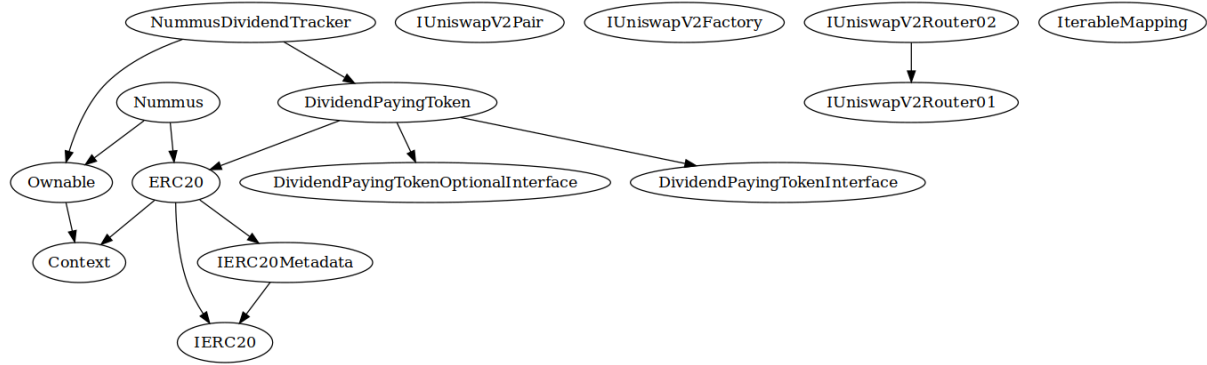
		Public	✓	ERC20
	decimals	Public		-
		External	Payable	-
	enableTrading	External	✓	onlyOwner
	setPresaleWallet	External	✓	onlyOwner
	setExcludeFees	Public	✓	onlyOwner
	setExcludeDividends	Public	✓	onlyOwner
	setIncludeDividends	Public	✓	onlyOwner
	setCanTransferBefore	External	✓	onlyOwner
	setLimitsInEffect	External	✓	onlyOwner
	setGasPriceLimit	External	✓	onlyOwner
	setcooldowntimer	External	✓	onlyOwner
	setMaxWallet	External	✓	onlyOwner
	setSwapTriggerAmount	Public	✓	onlyOwner
	enableSwapAndLiquify	Public	✓	onlyOwner
	setAutomatedMarketMakerPair	Public	✓	onlyOwner
	setAllowCustomTokens	Public	✓	onlyOwner
	setAllowAutoReinvest	Public	✓	onlyOwner
	_setAutomatedMarketMakerPair	Private	✓	
	updateGasForProcessing	Public	✓	onlyOwner
	transferAdmin	Public	✓	onlyOwner
	updateTransferFee	Public	✓	onlyOwner
	updateFees	Public	✓	onlyOwner

	getTotalDividendsDistributed	External		-
	isExcludedFromFees	Public		-
	withdrawableDividendOf	Public		-
	dividendTokenBalanceOf	Public		-
	getAccountDividendsInfo	External		-
	getAccountDividendsInfoAtIndex	External		-
	processDividendTracker	External	✓	-
	claim	External	✓	-
	getLastProcessedIndex	External		-
	getNumberOfDividendTokenHolders	External		-
	setAutoClaim	External	✓	-
	setReinvest	External	✓	-
	setDividendsPaused	External	✓	onlyOwner
	isExcludedFromAutoClaim	External		-
	isReinvest	External		-
	_transfer	Internal	✓	
	swapAndLiquify	Private	✓	
	swapTokensForEth	Private	✓	
	updatePayoutToken	Public	✓	onlyOwner
	getPayoutToken	Public		-
	setMinimumTokenBalanceForAutoDividends	Public	✓	onlyOwner
	setMinimumTokenBalanceForDividends	Public	✓	onlyOwner
	addLiquidity	Private	✓	

	forceSwapAndSendDividends	Public	✓	onlyOwner
	swapAndSendDividends	Private	✓	
NummusDividendTracker	Implementation	DividendPayingToken, Ownable		
		Public	✓	DividendPayingToken
	decimals	Public		-
	name	Public		-
	symbol	Public		-
	_transfer	Internal		
	withdrawDividend	Public		-
	isExcludedFromAutoClaim	External		onlyOwner
	isReinvest	External		onlyOwner
	setAllowCustomTokens	External	✓	onlyOwner
	setAllowAutoReinvest	External	✓	onlyOwner
	excludeFromDividends	External	✓	onlyOwner
	includeFromDividends	External	✓	onlyOwner
	setAutoClaim	External	✓	onlyOwner
	setReinvest	External	✓	onlyOwner
	setMinimumTokenBalanceForAutoDividends	External	✓	onlyOwner
	setMinimumTokenBalanceForDividends	External	✓	onlyOwner
	setDividendsPaused	External	✓	onlyOwner
	getLastProcessedIndex	External		-

	getNumberOfTokenHolders	External		-
	getAccount	Public		-
	getAccountAtIndex	Public		-
	setBalance	External	✓	onlyOwner
	process	Public	✓	-
	processAccount	Public	✓	onlyOwner
	updateUniswapV2Router	Public	✓	onlyOwner
	updatePayoutToken	Public	✓	onlyOwner
	getPayoutToken	Public		-
	_reinvestDividendOfUser	Private	✓	
	_withdrawDividendOfUser	Internal	✓	

Inheritance Graph



Flow Graph

You can find Flow graph image in the link below:

[Flow Graph](#)

Summary

Nummus Community contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like stop transactions.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io